FPS Mobile - Kotlin & Java - Android

1.1 Project

FPS Mobile Integration for Mobile Devices(Android/IOS)

1.2 Description

FPS Mobile Integration system is designed to identify and mitigate fraudulent activities carried out by malicious users on mobile devices. The system leverages advanced algorithms, data analysis, and behavioral tracking to detect unusual patterns that deviate from legitimate user behavior.

1.3 Revision History

Date	Comment	Author

Contents

FPS Mobile - Kotlin & Java - Android
- 1.1 Project
- 1.2 Description
- 1.3 Revision History

Section 2 - Overview
- 2.1 Purpose
- 2.2 Scope

Section 3 - System Architecture

Section 4 - SDK Sequence Diagram

Section 5 - Integration Guide
- 5.1 Kotlin + Java
  - 5.1.1 Dependencies
  - 5.1.2 SDK methods
    - 5.1.2.1 onCheckFPSTrans
    - 5.1.2.2 onResetFPSLastLiveTime
  - 5.1.3 Example with FpsService

Section 6 – FPS Gateway API
- 6.1 Base URL
- 6.2 Endpoint
- 6.4 Header
- 6.3 Body
- 6.4 Response

Section 8 – Glossary

Section 2 - Overview

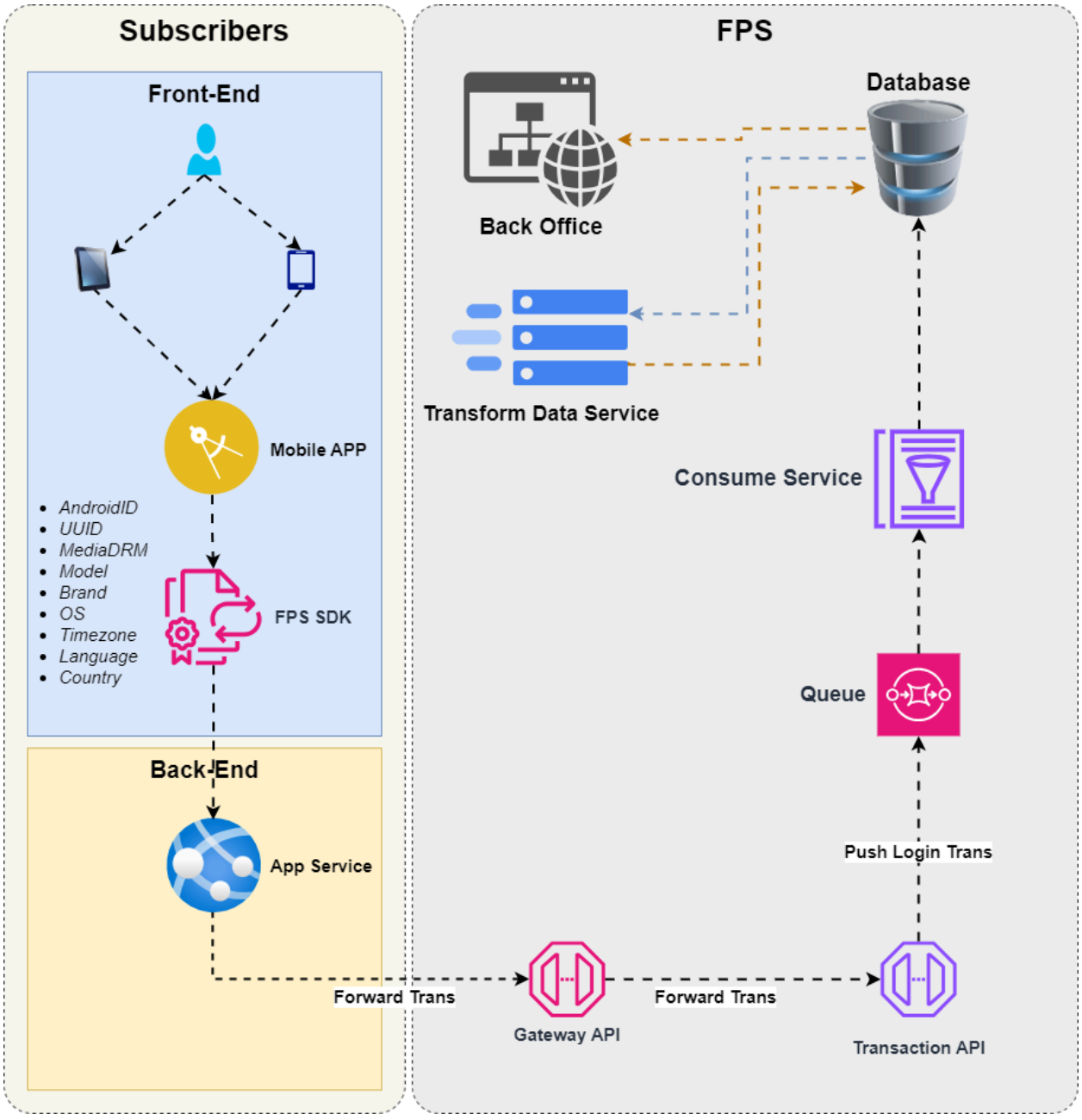
2.1 Purpose

FPS Mobile SDK is designed to integrate with mobile applications, enabling the detection of fraudulent activities by collecting device data and monitoring user behavior. This information is sent to the backend for periodic analysis, helping identify and prevent potential fraud without requiring real-time processing.

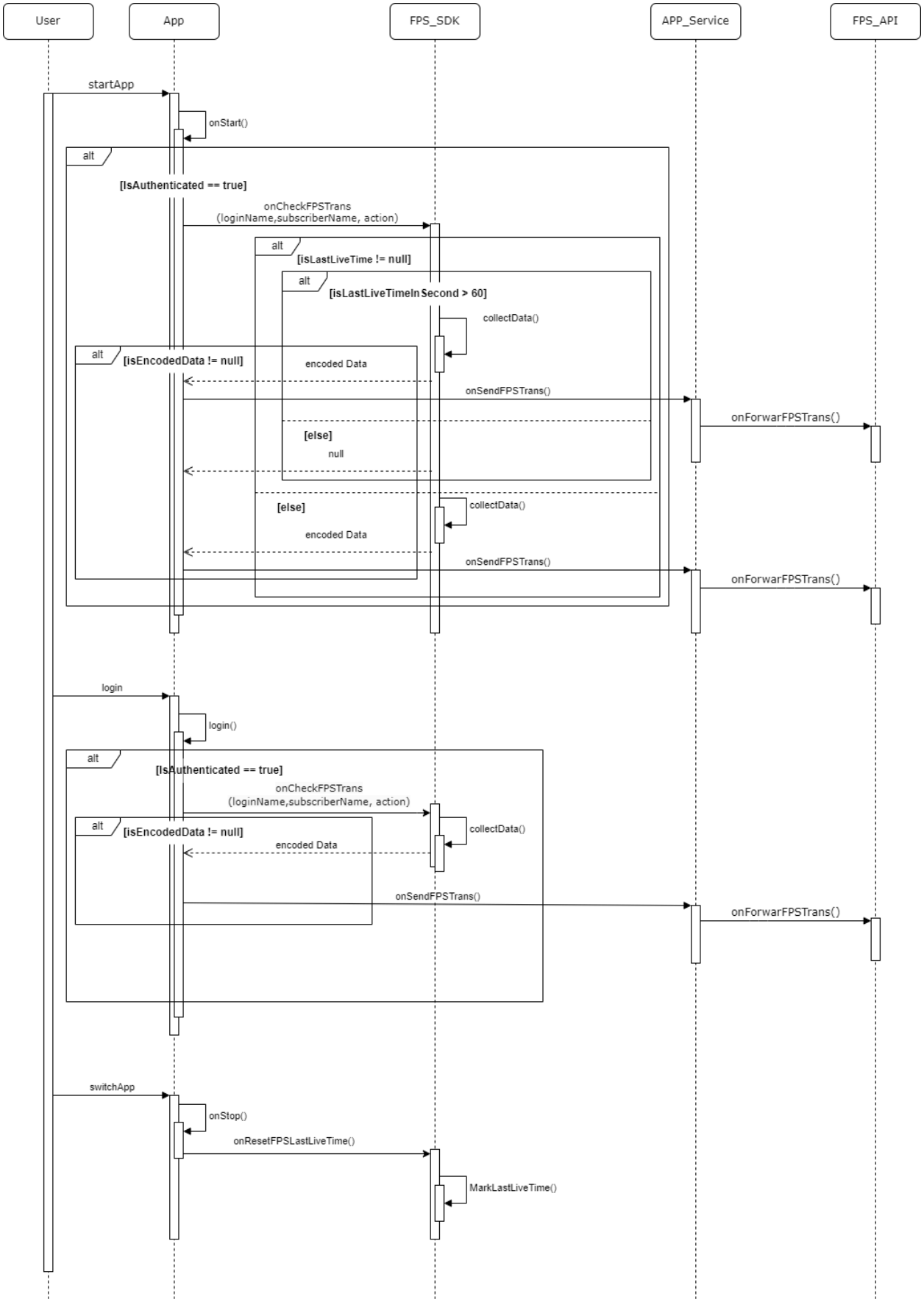
2.2 Scope

Seamlessly integrate into Android and iOS applications without impacting user experience

Section 3 - System Architecture



Section 4 - SDK Sequence Diagram


User
App
FPS_SDK
APP_Service
FPS_API
startApp
onStart()
alt
[IsAuthenticated == true]
onCheckFPSTrans
(loginName,subscriberName, action)
alt
[isLastLiveTime != null]
alt
[isLastLiveTimeInSecond > 60]
collectData()
alt
[isEncodedData != null]
encoded Data
onSendFPSTrans()
onForwarFPSTrans()
[else]
null
[else]
collectData()
encoded Data
onSendFPSTrans()
onForwarFPSTrans()
login
login()
alt
[IsAuthenticated == true]
onCheckFPSTrans
(loginName,subscriberName, action)
alt
[isEncodedData != null]
collectData()
encoded Data
onSendFPSTrans()
onForwarFPSTrans()
switchApp
onStop()
onResetFPSLastLiveTime()
MarkLastLiveTime()

Section 5 - Integration Guide

5.1 Kotlin + Java

This is applied for Kotline & Java language for Android.

5.1.1 Dependencies

There is a "**libs**" folder, attached to this Document. This folder contains:

```
+   com.fps.android.sdk.aar
```

Step 1: Copy this folder to the project root folder
Step 2: Add dependencies in **build.gradle.kts** by

```
dependencies {

    ...
    implementation(files("libs/com.fps.android.sdk.aar"))
}
```

Step 3: Sync the project

5.1.2 SDK methods

5.1.2.1 onCheckFPSTrans

This method is going to collect device information & return an encoded message, which would **be forwarded to the FPS via App Service**. It should be run in asynchronous IO, preventing from blocking the User Experience.

Method Signature

```
onCheckFPSTrans(context, subscriber, username, action)
```

Returns

String?

Parameters

Name	Type	Description	
context	Context	Application Context	
subscriber	String	The licensee's name was provided by FPS team	Maxbet
username	String	The username that the client uses to log in	test001
action	String	The action where fps	login, deposit,

		integrated	withdraw, etc…

5.1.2.2 onResetFPSLastLiveTime

This method would help FPS get more precious transactions. It should be called in the **onStop (Lifecycle Activity) method** when the user navigates to another application.

Method Signature

```
onResetFPSLastLiveTime()
```

Returns

Unit

5.1.3 Example with FpsService

FPS Service is implemented to collect and send mobile information to FPS gateway API.

- Initial FpsService using retrofit

```
interface IApiService {
    @Headers("Content-Type: application/json")
    @POST
    suspend fun send(@Url endpoint: String, @Body body: String): String
}

class FpsService(
    baseUrl: String,
    private val endpoint: String
) {
    private val retrofit: Retrofit = Retrofit.Builder()
        .addConverterFactory(ScalarsConverterFactory.create())
        .baseUrl(baseUrl)
        .build()

    private val apiService: IApiService =
retrofit.create(IApiService::class.java)

    fun checkTransaction(
        context: Context,
        subscriber: String,
        username: String,
        action: String
    ) {
        CoroutineScope(Dispatchers.IO).launch {
            try {
                val fpsTrans = FPS.onCheckFPSTrans(context, subscriber,
username, action)

                if(fpsTrans != null) {
                    val jsonObject = JSONObject()
                    jsonObject.put("transaction", fpsTrans)

                    apiService.send(endpoint, jsonObject.toString())
                }
            }
            catch (e: HttpException) {
                Log.d("HttpException", e.message!!)
            }
            catch (e: Exception) {
                Log.d("ApiService", e.message!!)
            }

        }
    }
}
```

- FpsService should be called in **login methods** and **onStart (Android Lifecycle Activity) method**

```
fun login(
   context: Context
) {

   ...

   val fpsService = FpsService("APP_SERVICE_BASE_URL", "APP_SERVICE_END_POINT")

   fpsService.checkTransaction(context, subscriber, username, "login")
}
```

```
class MainActivity : ComponentActivity() {

   override fun onStart() {
       super.onStart()

       ...

       val model: LoginViewModel by viewModels()
       if(model.isAuthenticated()) {
           val username = model.getUsername()

           val fpsService = FpsService("APP_SERVICE_BASE_URL",
"APP_SERVICE_END_POINT")

           fpsService.checkTransaction(this, subscriber, username, "login")
       }

   }

   override fun onStop() {
       super.onStop()

       ...

       FPS.onResetFPSLastLiveTime()
   }
}
```

Section 6 – FPS Gateway API

This is API to post the transaction to FPS

6.1 Base URL

- External network: `https://api.detecas.com`
- Internal network: `http://fpsapi.nebula.prod`

6.2 Endpoint

```
POST /FPS/MobileCheckTransaction
```

6.4 Header

Name	Description
Fps-Forwarded-For	The user's IP that is captured from the App Service

6.3 Body

Content-Type: application/json

Name	Description
transaction	The encoded message that is returned from the SDK

6.4 Response

Status	Description
200	**OK**
400	**Bad Request** The request must be corrected or added to the required parameters.
404	**Not Found** The requested resource was not found.
500	**Internal Server Error** An unexpected error occurred on the server.

Section 8 – Glossary